Concurrencia sobre STM32: del super-loop bloqueante a las interrupciones

Laboratorio 2 | Sistemas Electrónicos de Instrumentación

Pedro Villar | 25 de octubre de 2026

Resolví los seis desafíos sobre una **Nucleo-F411RE** en lugar de la F411E-DISCO de la guía. El LED verde es el de la placa (PA5) y el naranja, rojo y azul los cableé en un protoboard (PA6, PA7 y PB6). El botón es el de la placa (PC13) y es activo en bajo, por eso en el código "presionado" se compara contra `GPIO_PIN_RESET`. El reloj es el HSI de 16 MHz sin PLL: con prescaler de 16000 los timers cuentan a 1 kHz y el período en ms se escribe directo en el ARR. Acá incluyo solo el lazo principal y los callbacks de cada desafío; el proyecto completo está en el repositorio.

Desafío 1: Memoria de estado y antirrebote bloqueante

```
while (1) {
  if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) {    // presionado
    HAL_Delay(50);                                          // antirrebote bloqueante
    if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) {  // era real
      HAL_GPIO_TogglePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin);
      while (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET);
    }
  }
}
```

Lo importante acá es detectar el flanco y no el nivel: sin el `while` del final, mantener el botón apretado haría toggles sin parar. El rebote se filtra esperando 50 ms y volviendo a leer. El problema es que mientras espera, el procesador no puede hacer nada más. Funciona bien solo porque es la única tarea del programa.

Desafío 2: Temporización cooperativa con `HAL_GetTick()`

```
while (1) {
  uint32_t millisActual = HAL_GetTick();
  if (millisActual - millisAnterior >= 250) {            // ¿le toca al LED azul?
    millisAnterior = millisActual;
    HAL_GPIO_TogglePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin);
  }
  if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) // se lee siempre
    HAL_GPIO_WritePin(LED_VERDE_GPIO_Port, LED_VERDE_Pin, GPIO_PIN_SET);
  else
    HAL_GPIO_WritePin(LED_VERDE_GPIO_Port, LED_VERDE_Pin, GPIO_PIN_RESET);
}
```

En vez de frenar 250 ms, el lazo pregunta en cada vuelta si ya pasó el tiempo. Como el loop gira miles de veces por segundo, el verde responde al botón al instante y el azul mantiene su ritmo. La resta entre `uint32_t` sigue dando bien aunque el contador de ticks desborde. Ninguna tarea bloquea a la otra: eso es concurrencia cooperativa.

Desafío 3: Máquina de estados

```
while (1) {
  GPIO_PinState botonActual = HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin);
  if (botonActual == GPIO_PIN_RESET && botonAnterior == GPIO_PIN_SET) { // flanco
    HAL_Delay(20);                          // antirrebote corto, permitido en la transición
    estado = (estado + 1) % 3;              // 0 apagado -> 1 lento -> 2 rápido -> 0 ...
    ApagarTodosLosLeds();
  }
  botonAnterior = botonActual;

  if (estado == 1) periodo = 500;           // 1 Hz
  if (estado == 2) periodo = 100;           // 5 Hz
  if (estado != 0 && HAL_GetTick() - millisAnterior >= periodo) {
    millisAnterior = HAL_GetTick();
    InvertirTodosLosLeds();                 // toggle de los cuatro LEDs
  }
}
```

Acá se juntan las dos técnicas anteriores. El flanco se detecta comparando la lectura actual del botón con la anterior, y la variable `estado` guarda en qué modo está el sistema y define el período de

parpadeo. Como nada bloquea, el botón responde al instante sin importar qué tan rápido parpadeen los LEDs.

Desafío 4: Timer de hardware por polling

```
HAL_TIM_Base_Start(&htim2);              // desborda cada 100 ms (PSC 16000-1)
while (1) {
  if (__HAL_TIM_GET_FLAG(&htim2, TIM_FLAG_UPDATE) != RESET) {
    __HAL_TIM_CLEAR_FLAG(&htim2, TIM_FLAG_UPDATE);
    AvanzarSecuenciaLEDs();              // V-N-R-A-R-N, un paso por desborde
  }
}
```

Ahora el tiempo lo cuenta un timer de hardware en vez del procesador, con la precisión de un periférico dedicado e independiente del SysTick. Pero todavía hay que preguntar por el flag: si el lazo estuviera ocupado en otra cosa, la secuencia se atrasaría. Se gana precisión respecto al desafío 2, pero la limitación sigue siendo el polling.

Desafío 5: Interrupciones

```
HAL_TIM_Base_Start_IT(&htim3);           // TIM3 interrumpe cada 500 ms
while (1) {
  HAL_GPIO_TogglePin(LED_VERDE_GPIO_Port, LED_VERDE_Pin); // se nota lo lento del main
  HAL_Delay(5000);                       // main bloqueado 5 segundos
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM3)
    HAL_GPIO_TogglePin(LED_NARANJA_GPIO_Port, LED_NARANJA_Pin);
}
```

El main está bloqueado 5 segundos en un `HAL_Delay` (el verde lo delata, cambia cada 5 segundos) y aun así el naranja parpadea a 1 Hz exacto. El NVIC frena al main donde esté, ejecuta el callback y le devuelve el control. Ya nadie pregunta por un flag: el hardware avisa. Esto es concurrencia preemptiva.

Desafío 6: Antirrebote por muestreo en ISR

```
volatile uint8_t bandera_boton = 0;      // la escribe la interrupción

while (1) {                              // el main solo mira la bandera
  if (bandera_boton) {
    bandera_boton = 0;
    HAL_GPIO_TogglePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin);
  }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM4) {          // cada 10 ms
    static uint8_t contador = 0;
    if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) {
      contador++;
      if (contador == 5) bandera_boton = 1;   // 5 lecturas x 10 ms = 50 ms estable
    } else {
      contador = 0;                      // ruido o botón suelto: arrancar de nuevo
    }
  }
}
```

Es el mismo problema del desafío 1 pero resuelto sin bloquear: el TIM4 muestrea el botón cada 10 ms y recién con 5 lecturas seguidas da la presión por válida, porque el ruido del rebote no llega a juntar 5. El main y la interrupción se comunican con una bandera compartida, y ahí el `volatile` es obligatorio: sin él, el compilador asume que la variable nunca cambia y el toggle no ocurre.

Conclusiones

A lo largo de los desafíos, el manejo del tiempo fue pasando del procesador a los periféricos. Primero el procesador espera (`HAL_Delay`), después el lazo consulta la hora (`HAL_GetTick`), después un timer cuenta y el lazo pregunta por el flag, y al final el timer directamente interrumpe. Cada

paso libera más al main, pero el programa se vuelve más difícil de seguir y aparecen detalles nuevos, como el `volatile` en variables compartidas con una ISR. La regla que me llevo es simple: bloquear solo está bien cuando no hay nada más que atender, y casi siempre hay algo más que atender. Comparado con el laboratorio 1, donde cada bloque de la FPGA funcionaba en paralelo de verdad, acá hay un solo hilo que tiene que repartirse entre todo, y las interrupciones son la herramienta que hace que eso parezca paralelo.